```
    child2[split_pt:] = x1[split_pt:]
    c1 = {'params':child1, 'fitness': 0.0}     5
    c2 = {'params':child2, 'fitness': 0.0}
    return c1, c2
```

- ***1*** x1 and x2 are agents, which are dictionaries.
- ***2*** Extracts just the parameter vector
- ***3*** Randomly produces a split or crossover point
- ***4*** The first child is produced by taking the first segment of parent 1 and the second segment of parent 2.
- ***5*** Creates new child agents by packaging the new parameter vectors into dictionaries

This function takes two agents who serve as parents and produces two children or offspring. It does so by taking a random split or crossover point, and then taking the first piece of parent 1 and combining it with the second piece of parent 2, and likewise combines the second piece of parent 1 and the first piece of parent 2. This is exactly the same mechanism we used to recombine strings before.

That was the first stage for populating the next generation; the second stage is to mutate the individuals with some fairly low probability. Mutation is the only source of new genetic information in each generation—recombination only shuffles around information that already exists.

Listing 6.9. Mutating the parameter vectors

```
def mutate(x, rate=0.01):
1
    x_ = x['params']
    num_to_change = int(rate * x_.shape[0])
2
    idx = np.random.randint(low=0,high=x_.shape[0],size=
(num_to_change,))
    x_[idx] = torch.randn(num_to_change) / 10.0
3
    x['params'] = x_
    return x
```

- ***1*** rate is the mutation rate, where 0.01 is a 1% mutation rate.
- ***2*** Uses the mutation rate to decide how many elements in the parameter vector to mutate
- ***3*** Randomly resets the selected elements in the parameter vector